Top 50 Tricky SQL Theoretical Interview Questions

SQL Basics

1. What is the difference between the `WHERE` clause and the `HAVING` clause, and when should each be used?

Answer

---

2. What are the differences between `DELETE`, `TRUNCATE`, and `DROP` commands in SQL?

Answer

3. Why does SQL treat `NULL` differently from other values, and what does it actually represent?

Answer

4. Why does the expression `NULL = NULL` not evaluate to `TRUE` in SQL?

Answer

5. What is the difference between `COUNT(*)`, `COUNT(column_name)`, and `COUNT(DISTINCT column_name)`?

Answer

6. What is the difference between `CHAR` and `VARCHAR`, and when would you choose one over the other?

Answer

7. What is the difference between a primary key, a unique key, and a foreign key?

Answer

8. Can a table have multiple unique keys and multiple foreign keys, and why?

Answer

9. What are candidate keys and alternate keys, and how are they different from primary keys?

Answer

10. Why is SQL known as a declarative language instead of a procedural language?

Answer

Joins

11. What is the difference between `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, and `FULL OUTER JOIN`?

Answer

12. Why do joins sometimes produce duplicate rows even when the data appears correct?

Answer

13. What is the difference between `EXISTS` and `INNER JOIN`, and when is one preferred over the other?

Answer

14. What is the difference between `LEFT JOIN` with `IS NULL` and `NOT EXISTS` for finding unmatched records?

Answer

15. Under what circumstances can a Cartesian product occur, and how can it be prevented?

Answer

16. Why is it important to specify the correct join condition while joining multiple tables?

Answer

17. Can a query contain multiple joins to the same table, and what are common use cases for doing so?

Answer

18. What is a self join, and in what situations is it commonly used?

Answer

GROUP BY and Aggregation

19. Why must every non-aggregated column in the `SELECT` list appear in the `GROUP BY` clause?

Answer

20. What is the difference between `GROUP BY` and `DISTINCT`, even though both remove duplicate values?

Answer

21. How does SQL handle `NULL` values while performing aggregate functions?

Answer

22. What happens when aggregate functions are used without a `GROUP BY` clause?

Answer

23. Why does `COUNT(column_name)` ignore `NULL` values while `COUNT(*)` counts every row?

Answer

24. Can the `HAVING` clause be used without a `GROUP BY` clause, and how does SQL process such a query?

Answer

Window Functions

25. What is the purpose of window functions, and how are they different from aggregate functions?

Answer

26. What is the difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`?

Answer

27. Why can't window functions be directly used in the `WHERE` clause?

Answer

28. What is the difference between `PARTITION BY` and `GROUP BY`?

Answer

29. In what scenarios are `LAG()` and `LEAD()` functions commonly used?

Answer

30. How do window functions improve query readability compared to correlated subqueries?

Answer

EXISTS, IN and Subqueries

31. What is the difference between `IN` and `EXISTS`, and how does performance differ for large datasets?

Answer

32. Why does `NOT IN` sometimes return unexpected results when `NULL` values are present?

Answer

33. What is the difference between a correlated subquery and a non-correlated subquery?

Answer

34. What are the advantages and disadvantages of using Common Table Expressions (CTEs) instead of subqueries?

Answer

35. What is a recursive Common Table Expression (CTE), and where is it commonly used?

Answer

36. What is the difference between a CTE, a temporary table, and a derived table?

Answer

Indexes and Performance

37. What is an index, and how does it improve query performance?

Answer

38. What is the difference between clustered and non-clustered indexes?

Answer

39. Why can indexes improve `SELECT` queries but slow down `INSERT`, `UPDATE`, and `DELETE` operations?

Answer

40. Why should indexes not be created on every column in a table?

Answer

41. What is a composite index, and why does column order matter in a composite index?

Answer

42. What is index selectivity, and why is it important for query optimization?

Answer

43. Why does the database optimizer sometimes ignore an available index and perform a full table scan instead?

Transactions and Concurrency

44. What are the ACID properties of a transaction, and why are they important in database systems?

Answer

45. What is the difference between `COMMIT` and `ROLLBACK` in SQL transactions?

Answer

46. What are transaction isolation levels, and how do they affect data consistency and concurrency?

Answer

47. What are dirty reads, non-repeatable reads, and phantom reads, and how are they prevented?

Answer

48. What is a deadlock, and how does a database detect and resolve it?

Answer

Advanced SQL

49. Why can two SQL queries that return the same result have significantly different execution times?

Answer

50. What are the most common reasons for poor SQL query performance, and how would you identify and resolve them?

Answer

Bonus: Frequently Asked "Why" Questions

*** Why is `SELECT *` considered a bad practice in production environments?**

Answer

*** Why is `UNION ALL` generally faster than `UNION`?**

Answer

*** Why is `ORDER BY` not guaranteed unless explicitly specified?**

Answer

*** Why is `DISTINCT` considered an expensive operation on large datasets?**

Answer

*** Why is normalization important, and when should denormalization be considered?**

Answer

*** Why is the `EXPLAIN` or execution plan one of the most important tools for SQL performance tuning?**

Answer

*** Why is `EXISTS` often preferred over `IN` for large subqueries?**

Answer

*** Why are statistics important for the SQL query optimizer?**

Answer

*** Why do databases use B-Tree indexes by default instead of other data structures?**

Answer

*** Why does the order of conditions in a `WHERE` clause usually not affect query performance, even though many developers believe it does?**

Answer